“© 2023 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.”

Multi-Stream Concept Drift Self-Adaptation Using Graph Neural Network

Ming Zhou, Jie Lu*, *Fellow, IEEE,* Yiliao Song, *Member, IEEE* and Guangquan Zhang

Abstract—Concept drift is the phenomenon where the data distribution in a data stream changes over time. It is a ubiquitous problem in the real-world, for example, a traffic accident would cause a jam in a certain period, leading to a distribution change in traffic speed. Most research in the concept drift field focuses on single data stream, however, few of them consider multi-stream environments which are more in line with the application needs. To fill this gap, we propose a multi-stream prediction setting and a multi-stream concept drift self-adaptation framework using graph neural network, named SAGN. In SAGN, we reconsider the learning procedure of GNN-based predictors from an aspect of concept drift adaptation for multi-stream. By this design, the prediction task is converted into online streaming data tasks in sub-graphs. Each sub-graph corresponds to an adaptation target and will be updated over time. In this way, locally we can overcome drift in each sub-graph by a designed adaptation technique, and globally the correlation between different data streams is well-preserved as a graph structure. Therefore, whether drift occurs or not, in one or several streams, SAGN can provide consistently accurate prediction results. We comprehensively tested SAGN on both synthetic and real-world, drift and non-drift data in the multi-step prediction task. The experiment results show that SAGN is able to achieve state-of-the-art performance in most cases.

Index Terms—Concept drift, Drift adaptation, Multiple data streams, Graph neural network.

1 Introduction

The explosion in the number of data sources generates fast-evolving data streams which have become one of the main data forms in current daily life [1]. Concept drift is a ubiquitous phenomenon in streaming scenarios. It refers to changes of data distribution in the data stream over time [2]. For example, a traffic accident causes distribution changes of traffic speed; topic changes cause distribution changes of the text in the media stream. Concept drift brings challenges to traditional machine learning algorithms as these algorithms assume the upcoming data has the same distribution as the historical data [3]. Such static learning algorithms will be invalid in this dynamically changing environment. When drift occurs, the learned model based on historical data cannot adapt to new data distribution and the prediction results will no longer be reliable [4].

Concept drift adaptation is to handle the concept drift problem and there are two main categories: informed and blind adaptation [5]. The informed adaptation, when to update the predictor is determined by the feedback from a drift detection module [6]. For example, [7] use a sliding window to detect the drift and the predictor is updated when drift is detected. For blind methods, regardless of whether drift occurs, the model will be updated according to pre-designed rules, such as incremental learning-based adaptation methods [8], [9].

So far, most existing adaptation methods are designed for single data stream [10]. In practice, however, multi-stream is more common in most real scenarios [11]. If we treat each stream in multi-stream independently and simply apply the adaptation methods for single stream on them separately, the correlation between streams would be ignored [12]. As a result, the prediction results are less appropriate to support data-driven decision-making for multiple correlated streams.

Multi-stream environments have complex drift situations due to latent correlations between streams. With the consideration of possible concept drift issues in multi-stream, the correlation between data streams is more complex as well [13]. For example, when drift occurs on a data stream, other streams may soon or have faced a similar drift problem, namely, drifts may appear in multiple streams simultaneously or with a delay [14]. This is called a drift correlation between streams [10]. Constructing a drift adaptation framework for multiple correlated and drift-correlated streams has become a new challenge in the concept drift field. Several researchers have proposed some solutions [15]. For example, [16] proposes an adaptive framework for multi-stream classification, which sets the drift detection module on the source stream to update the classifier of the target stream. There is a similar stream setting in [17], [18]. The work in [10] focuses on the concurrent drift problem in multi-stream and proposes the MuNet framework which builds Bayes-based connectors between streams.

Current studies in multi-stream still have many deficiencies. First, some of them assume a source and target stream in their setting [17], [18]. However, in most cases, there are no explicit source and target streams in real-world multi-stream. As a result, these methods are less applicable in large-scale multi-stream environments. Besides, current methods do not consider stream dependencies in-depth, or they only consider specific drift types.

To fill those gaps, we propose a self-adaptation learning framework for multi-stream using graph neural network (GNN), named SAGN. SAGN can handle different types of

*Corresponding author. The authors are with the Decision Systems and e-Service Intelligence Laboratory, Australian Artificial Intelligence Institute (AAII), Faculty of Engineering and Information Technology, University of Technology Sydney, Ultimo, NSW 2007, Australia. (Ming.Zhou@student.uts.edu.au, {Jie.Lu, Yiliao.Song, Guangquan.Zhang}@uts.edu.au)

drift in multi-stream. It does not require the pre-knowledge of which streams are source or target. In SAGN, the dependencies between streams are presented in a graph structure by introducing GNN. GNN is well-known for deeply mining the correlations and dependencies among multiple nodes in the graph structure [19], but it is not originally designed to overcome the concept drift problem. To solve this issue, we reconsider the learning procedure of GNN-based methods from the aspect of concept drift adaptation. We convert the original prediction task into online streaming data tasks in sub-graphs. Each sub-graph handles concept drift that corresponds to an adaptation target locally. Globally, the entire graph presents the correlation between streams. Therefore, SAGN can solve the concept drift with considering the dependencies between streams.

To implement SAGN, we design an offline training stage and an online testing stage. The offline training stage assumes a fixed batch of historical data is available to train an initial model. In the online testing stage, new samples arrive one by one over time, and we update the initial model when every new sample arrives. The offline training uses historical data and Gumbel sampling to train an initial GNN. This GNN will be continuously updated (via sub-graphs) to adapt concept drift in the online testing stage. Instead of using one GNN, we choose the best GNN from a pool of GNNs. The GNN pool is a collection of multiple well-performed GNNs. The best GNN at each update is selected by testing with a dynamically updated validation set.

Our contributions are as follows:

- We propose a multi-stream prediction setting that multiple streaming data, containing multiple correlated streams, share the same time step. The correlation in multi-stream contains the dependencies between streams and the correlation between drifts. Compared to previous research, it does not need to assume the source/target stream.
- We design an online adaptation strategy for the GNN to handle different types of concept drift which is implemented locally in sub-graphs. An advantage of this design is that globally it maintains the correlation in multi-stream. The proposed dynamic validation set and GNN pool rolling mechanism are to work in tandem to achieve this process.
- We propose SAGN, a multi-stream concept drift self-adaptation framework using GNN. SAGN can overcome the concept drift problem in multi-stream with preserving the correlation between streams. Compared to existing methods, it is more applicable for large-scale multi-stream environments.
- We tested SAGN comprehensively on synthetic and real-world, drift or non-drift data. The experiment results provide a useful guide for future applications or future studies in this area.

The rest of this paper is organized as follows. In Section 2, the concept drift adaptation methods and the Graph Neural Network are reviewed. The concept drift problem in multi-stream and related preliminaries settings are formulated in Section 3. Motivated by the challenges in the current research, our proposed solution SAGN is introduced in detail in Section 4. In section 5, we evaluate our solution using synthetic and real-world data sets. We compare the prediction results of the baselines, which include traditional methods, the adaptation method, and the latest GNN-based methods. Finally, Section 6 concludes our work and discusses future study.

2 Related Work

2.1 Concept Drift Adaptation

Adjusting and updating the model to fit the new data distribution is the essence of developing an adaptive framework. Under the informed strategy, to find a suitable updated time point, the frameworks capture drift information by setting up certain drift detection modules. Correspondingly, there is no drift detection module in blind methods, and the models will be updated regularly according to pre-designed rules.

The sliding window that ADWIN uses is one of the classic methods in the informed strategy [20]. The defined validation statistic properties are combined with the variable window sizes to detect distribution changes over the data stream. The triggered threshold indicates that the learner needs to be updated. At present, the fuzzy window is becoming the new development opportunity for this strategy [21], [22]. The fuzzy window-based method further improves the flexibility of window adjustment. In addition, some appropriately expanded and improved machine learning methods have also achieved good performance. For example, FP-ELM [23] and DELM [24] both expand on the traditional Extreme Learning Machine (ELM) algorithm [25]. The former introduces the forgetting parameters to realize the adaptation process, while the latter achieves adaptation by adding some more nodes to the hidden layer.

In the blind strategy, a commonly used method is to update the individual classifier. VFDT [26] is a classic algorithm developed from the Hoeffding tree, and many follow-up algorithms were inspired by it, such as CVFDT [27], VFDTc [28]. In VFDT, the decision nodes that satisfy the conditions are selected by the Hoeffding inequality Within a certain degree of confidence, so as to adjust the model by dividing the feature attributes. The methods based on the classifier ensemble also have corresponding advantages in dealing with concept drift in some specific drift types [29]. Some of them introduce classic ensemble frameworks, including Bagging, Boosting, Random Forest, voting, and sampling, such as [8], [30]. There are also some methods that combine neural networks or fuzzy techniques to achieve good prediction performance, such as [31].

Most of the aforementioned methods are designed for single data stream. Applying these methods directly to multiple streams is inefficient and very difficult to scale. Also, the correlation between streams is ignored. A few researchers propose some solutions. [14] proposes a variance matrix to measure the similarity of drifts in different streams. [10] just focuses on the concurrent drift problem in multi-stream. These methods still have many defects such as they need to assume the source/target stream in advance or just focus on some specific drift types.

2.2 Graph Neural Network

GNN is a well-known efficient tool for mining the deep dependencies between objects in non-Euclidean domains. Although many objects are not graph-structured data, like time series, GNN still offers more possibilities to deeply mine the dependencies between multiple nodes and it has been used to help with prediction tasks in recent years.

It is common to construct distance-based spatial dependency structures on pre-defined graphs which contain the location information of sensors. DCRNN [32] is proposed for traffic streaming sequence data. The model builds a diffuse convolutional network on a graph structure. The bidirectional walk and predetermined sampling capture the spatial and temporal dependencies respectively. GraphWaveNet [33] calculates an efficient adjacency matrix on the pre-defined graph, by combining graph convolution and dilated casual convolution to extract the dependency information of nodes at different granular levels. The combination of attention mechanism and graph structure is applied in [34], multi spatio-temporal attention blocks are embedded in the encoder-decoder architecture. DGCRN [35] proposes the dynamic adjacency matrix as the product of the supernetwork generation which combines with the pre-defined graph to help prediction task. ASTGCN [36] proposes a latent network of Laplace matrix to represent the spatial-temporal connection of the traffic data adaptively.

Although pre-defined graphs can provide some prior knowledge, not all practical data has pre-defined graphs. The authors of [37] eschew the use of pre-defined graphs and designed two adaptive modules to learn multiple node parameters and data graphs separately. [38] proposes the GTS framework to use Gumbel distribution to simulate global distribution from historical data. These aforementioned methods achieve good prediction performance without using pre-defined graphs.

These GNN-based methods inspired us to construct spatio-temporal topologies from graph structures to mine potential dependencies between streams, but none of them considers the concept drift problem in multi-stream. The graph structures of these models are just constructed with the historical data distribution. In practical application scenarios, sudden or incremental drift changes are inevitable, such as sudden traffic accidents or extremely bad weather. The offline prediction mode does not take into account the complexity and instability of the streaming environment. This is also the key motivation for us to propose the multi-stream self-adaptation method under concept drift.

3 Problem Statement and Preliminaries

In this section, we first settle some important notations, definitions, and problem statements in Section 3.1. Also, we introduce the Gumbel-Softmax Trick method in Section 3.2 as the preliminaries that are used in our SAGN.

3.1 Problem Statement

In this section, the definitions and problem statement are introduced. We present a table of some important notations in Table 1 for brevity.

TABLE 1
Summary of Notations

Notations	Description
$\mathcal{S} = \{\mathcal{D}^1, \cdots, \mathcal{D}^s\}$	A multi-stream contains s data streams.
$\mathcal{D} = \{T^1, \cdots, T^g\}$	A data stream contains g time series.
$T^j = \{\boldsymbol{X}_t^j\}$	A time series in the data stream.
$\boldsymbol{X}^j$	The feature value of the time series.
$t \in \mathbb{Z}^+$	The time step.
$\mathcal{P}$	The probability distribution.
p	The probability density function.
τ	A time period.
Δ	The past window size.
η	The prediction time steps.
$\mathcal{G}$	The correlation graph structure of multi-stream.
H	The predictor for multi-stream.
$\mathcal{S}^{tr}, \mathcal{S}^{val}, \mathcal{S}^{te}$	The training, validation and testing sets.
$P = \{H_1^i \| i \in [C]\}$	The designed GNN pool with size C.
V^*	The designed dynamical validation set.
B_o	The size of V^*.

Definition 1 (Single Data Stream)**.** *A data stream is formulated by $\mathcal{D} = \{T^1, T^2, \cdots, T^g\}$, and it contains g time series. For any $j \in [g]$, $T^j = \{\boldsymbol{X}_t^j | t = 1, \cdots, \infty\}$ is one time series in the stream $\mathcal{D}$, where $\boldsymbol{X}_t^j$ represents the feature value of the T^g in time step t. $T^j \in \mathcal{D}, j \in [g]$ is generated from distribution $\mathcal{P}_t$ with probability density function denoted as $p_t(\boldsymbol{X}_t^j)$.*

Remark 1. *For example, consider a data stream generated by multiple sensors at a weather monitoring station. The time series produced by different sensors in this data stream share the same location information, and different time series represent different monitoring features.*

Definition 2 (Concept Drift in Single Data Stream [39])**.** *Concept drift occurs in a data stream $\mathcal{D}$, if $\exists t_{g^{(i)}}$ that*

$$\begin{cases} p_{t+1}(\mathcal{D}) \neq p_t(\mathcal{D}), \text{ for } t = t_{g^{(i)}} \\ p_{t+1}(\mathcal{D}) = p_t(\mathcal{D}), \text{ for } t \in \left(t_{g^{(i)}+\tau_i}, t_{g^{(i+1)}}\right) \end{cases}$$

where $\forall i,\ t_{g^{(i+1)}} - t_{g^{(i)}} > 1$, $t \in \mathbb{Z}^+$ represents the time step, $g^{(i)}$ represents the ith drifted time point, and $1 < \tau_i < t_{g^{(i+1)}} - t_{g^{(i)}}$ is specifically for the occurrence of incremental drift.

Remark 2. *A data stream contains concept drift if the data distribution changes at least once, namely $\left\{t_{g^{(i)}}\right\} \neq \emptyset$ that $p_{t+1}(\mathcal{D}) \neq p_t(\mathcal{D})$, for $t = t_{g^{(i)}}$; in addition, the changed pattern is not ephemeral, but will last for a period (at least last for two time steps), which is manifested by $\forall i,\ t_{g^{(i+1)}} - t_{g^{(i)}} > 1$. The pattern stays the same in this period that $p_{t+1}(\mathcal{D}) = p_t(\mathcal{D})$, for $t \in \left(t_{g^{(i)}+\tau_i}, t_{g^{(i+1)}}\right)$; here $\tau_i = 1$ when the drift occurs suddenly while $\tau_i > 1$ when the drift occurs incrementally in the period of $\left(t_{g^{(i)}+1}, t_{g^{(i)}+\tau_i}\right)$.*

Suppose $T^{j'} \in \mathcal{D}$ is the target series, the prediction task for $\mathcal{D}$ refers to the prediction for $T^{j'}$. For example, there are many feature series in a weather station that presents a data stream, if the 'humidity' series is the target series, the

prediction task for this stream refers to the prediction for the 'humidity' series.

Learning Aim at t-step for Single Data Stream: Let $\mathcal{D} = \{T^1, T^2, \cdots, T^g\}$ as a data stream, $T^{j'} = \{\boldsymbol{X}_t^{j'} | t = 1, \cdots, \infty\}$ as the target series. To obtain the prediction values of the next η time steps, the learning aim is to build a predictor H_t for $p_t(\mathcal{D})$ using the values in the past window of Δ steps, which can be denoted as

$$H_t = \arg\min_{h \in \mathcal{H}} \ell\left(h\left(\mathcal{D}_{t-\Delta+1:t}\right), \mathcal{D}_{t+1:t+\eta} | \mathcal{D} \in p_t(\mathcal{D})\right),$$

where $\mathcal{H}$ is the hypothesis set, $\ell : \mathbb{R}^1 \times \mathbb{R}^1 \to \mathbb{R}_+$ is the loss function used to measure the magnitude of the error.

Definition 3 (Multi-Stream). *Multi-stream is a collection of multiple single data streams, and can be formulated by $\mathcal{S} = \{\mathcal{D}^1, \mathcal{D}^2, \cdots, \mathcal{D}^s\}$. Every stream $\mathcal{D}^s = \{T^1, T^2, \cdots, T^{g_s}\}$ in $\mathcal{S}$ contains several time series. Each T denotes one feature series, and there are g_s different features in total.*

Remark 3. *Handling multi-stream problem is a big challenge. We propose to solve a multi-stream problem by each of the target features in the stream. Specifically, in this research, we assume that all streams in $\mathcal{S}$ contain the same features. This scenario is common in real applications. For example, many meteorological monitoring stations are set up in different city areas, where each station monitors multiple meteorological indicators (humidity, wind speed, temperature, etc.).*

Definition 4 (The Correlation in Multi-Stream). *The correlation in multi-stream includes the correlation between streams and a drift correlation that the occurrence of drift in one stream is related to the occurrence of drift in another stream. We use $\mathcal{G}_{true}$ to denote the correlation in multi-stream. For example, in a weather multi-stream, nearby weather stations share similar weather patterns. A sudden change of weather, for example, bushfire, will first lead to the drift of "humidity" in bushfire stations, and then other nearby stations will face a similar drift problem later.*

Remark 4. *In most practical situations, it is hard for us to obtain $\mathcal{G}_{true}$. Because it is never known in advance when drift occurs and how severe it will be. In addition, $\mathcal{G}_{true}$ may also change over time rather than maintaining a fixed structure. Therefore, we need to estimate $\mathcal{G}_{true}$ during the learning procedure in multi-stream.*

Definition 5 (Concept Drift in Multi-Stream). *Let $\mathcal{S} = \{\mathcal{D}^1, \mathcal{D}^2, \cdots, \mathcal{D}^s\}$ is a multi-stream. A multi-stream $\mathcal{S}$ has concept drift if any data stream in $\mathcal{S}$ has concept drift.*

Definition 6 (Concept Drift Correlation in Multi-stream). *Given $\mathcal{D}^u, \mathcal{D}^v \in \mathcal{S}$ (where $u \neq v$), p^u and p^v are the probability density function for stream $\mathcal{D}^u$ and $\mathcal{D}^v$, respectively. The drift correlation has two cases, concurrent drift and delayed drift.*

A concurrent drift occurs if $\exists t_{g^{(i)}}$ that

$$\begin{cases} p^u_{t+1} \neq p^u_t, p^v_{t+1} \neq p^v_t \text{ for } t = t_{g^{(i)}} \\ p^u_{t+1} = p^u_t, p^v_{t+1} = p^v_t \text{ for } t \in \left(t_{g^{(i)}+\tau_i}, t_{g^{(i+1)}}\right) \end{cases}$$

A delayed drift occurs if $\exists t_{g^{(i)}}, \exists t_{g^{(j)}}$ and $i \neq j$ that

$$\begin{cases} p^u_{t_i+1} \neq p^u_{t_i}, p^v_{t_j+1} \neq p^v_{t_j}, \text{ for } t_i = t_{g^{(i)}}, t_j = t_{g^{(j)}} \\ p^u_{t_i+1} = p^u_{t_i}, p^v_{t_j+1} = p^v_{t_j}, \text{ for } t_i \in \left(t_{g^{(i)}+\tau_i}, t_{g^{(i+1)}}\right), \\ \qquad\qquad t_j \in \left(t_{g^{(j)}+\tau_j}, t_{g^{(j+1)}}\right) \end{cases}$$

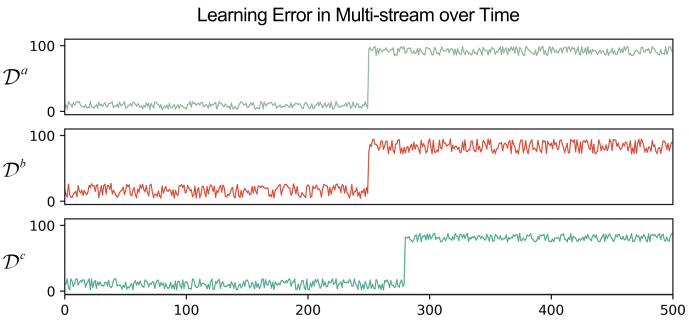


Fig. 1. An example of concept drift correlation in multi-stream. Concurrent drift occurs in $\mathcal{D}^a$ and $\mathcal{D}^b$ ($t = 250$), and delayed drift occurs in $\mathcal{D}^c$ ($t = 280$).

Remark 5. *Concurrent drift means drifts occur in two or more streams in the multi-stream at the same time. If drifts occur with a time difference, this situation refers to delayed drift. We use the learning error of static predictors to explicitly represent two cases. When drift occurs, the error rates increase significantly due to the inability of the static predictors to adapt to the new data distribution. As shown in Fig. 1, Clearly, concurrent drift occurs in $\mathcal{D}^a$, $\mathcal{D}^b$ at $t = 250$ while delayed drift occurs in $\mathcal{D}^c$ at $t = 280$.*

Remark 6. *It should also be noticed that the correlation in multi-stream $\mathcal{G}_{true}$ may change over time because of the occurrence of concept drift. For example, as shown in Fig. 2, the nodes $\mathcal{D}^1$, $\mathcal{D}^2$, $\mathcal{D}^3$, $\mathcal{D}^4$ and $\mathcal{D}^5$ represent five data streams in multi-stream separately and the edges in the graph structure represent correlations between two streams. It can be seen that the correlation remains stable globally, but if drift occurs in one of these streams at t step, such as $\mathcal{D}^4$, this drift may bring about changes in correlation locally, corresponding to changes in sub-graphs at $t + 1$ step.*

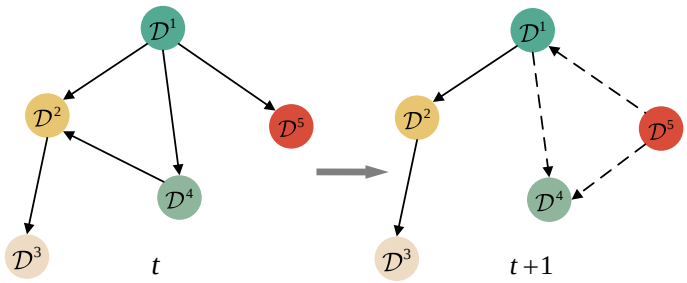


Fig. 2. Correlation in multi-stream changes locally from t to $t + 1$ due to the occurrence of concept drift.

Problem 1 (Learning for Multi-Stream at t-step with known $\mathcal{G}_{true}$). *To obtain the prediction values of the next η time steps, the learning aim is to build a predictor H_t for $p_t(\mathcal{S})$ using the values in the past window of Δ steps. The learning process is based on the correlation $\mathcal{G}_{true}$ of the multi-stream $\mathcal{S}$, which can be denoted as*

$$H_t = \arg\min_{h \in \mathcal{H}} \ell\left(h\left(\mathcal{S}_{t-\Delta+1:t}, \mathcal{G}_{true}\right), \mathcal{S}_{t+1:t+\eta}\right)$$

Remark 7. *According to Problem 1, the learning objective in a multi-stream setting is to construct the predictor at each time point under the correlation in streams.*

If we use single-stream concept drift adaptation methods for the learning in multi-stream directly. Namely, we consider each

stream separately and apply single-stream adaptation method one by one. The correlation $\mathcal{G}_{true}$ in multi-stream will be ignored. Besides, to obtain the prediction of s target features, we need to build s predictors for each separately. In this situation, Problem 1 is degenerated to

$$H_t^s = \underset{h^s \in \mathcal{H}^s}{\arg\min}\, \ell\left(h^s\left(D^s_{t-\Delta+1:t}\right), D^s_{t+1:t+\eta}\right).$$

Problem 1 represents an idealistic situation that $\mathcal{G}_{true}$ is known in advance. However, $\mathcal{G}_{true}$ is not available in most real cases. Therefore we propose to learn a correlation graph structure $\mathcal{G}_t$ to estimate $\mathcal{G}_{true}$. $\mathcal{G}_t$ is learned from the historical data to represent the correlation of the multi-stream $\mathcal{S}$. $\mathcal{G}_t$ will be continuously updated during the learning process. With considering the $\mathcal{G}_t$, the learning task has been changed, and the adaptation methods for single-stream are no longer applicable in this setting. To cover such a situation, we propose Problem 2.

Problem 2 (Learning for Multi-Stream at t-step with unknown $\mathcal{G}_{true}$)**.** *The learning aim for multi-stream at t-step with unknown $\mathcal{G}_{true}$ is to learn a correlation graph structure $\mathcal{G}_t$ and a predictor H_t for $p_t(\mathcal{S})$. The learning process is formalized as*

$$H_t, \mathcal{G}_t = \underset{h \in \mathcal{H}, \mathcal{G}}{\arg\min}\, \ell\left(h\left(\mathcal{S}_{t-\Delta+1:t}, \mathcal{G}_{t-1}\right), \mathcal{S}_{t+1:t+\eta}\right). \quad (1)$$

$\mathcal{G}_t$ is updated from $\mathcal{G}_{t-1}$ with every newly arrived sample in the online testing stage (Section 4.2). $\mathcal{G}_0$ is initialized on historical data using the Gumbel-Softmax trick (Section 3.2).

3.2 Gumbel-Softmax Trick

Although some methods [33], [37] are proposed to partially or completely avoid using the pre-defined graph, most of them still cannot achieve a high level of prediction accuracy without any pre-defined graph information. To calculate the correlation graph structure of multi-stream without using the pre-defined graph completely, we use the Gumbel-Softmax trick [40], [41] to learn a sampled graph structure based on the historical data. This subsection will introduce this technique in detail.

In a discrete data space, whether a dimension is selected or not depends on whether the cumulative weight on that dimension is the maximum. In a multi-dimensional vector $\Omega = [\alpha_1, \alpha_2, \alpha_3, ...\alpha_i], i \in [N]$, whether a dimension is selected depends on whether the weight of that dimension is maximum. However, difficulties remain between converting the vectors into a probability-based representation and retaining computable gradients.

Normalizing a multi-dimensional vector by connecting a softmax function is a common way of dealing with this type of problem. By connecting the softmax function, the multi-dimensional vector can be normalized and the gradient calculation can be realized, but more probability information is lost. Sampling according to the probability distribution cannot perform the gradient calculation to update the network parameters. The Gumbel-Softmax technique is proposed to overcome the inability to apply the reparameterization technique to discrete data, and two results have been argued: 1) The discrete distributions can be parameterized by the Gumbel distribution; 2) The function corresponding to the Gumbel distribution can be continuous, benefiting from the continuous approximation property of the temperature parameter δ. When δ is reduced to zero, it can represent discontinuous expressions of the original distribution.

To make a discrete probability distribution meaningful rather than simply taking the value with the highest probability, and to be able to compute the gradient. The Gumbel trick is processed as follows:

First, perturbed Gumbel noises based on uniform distribution are generated which correspond to N dimensional vectors Ω. $\{\mathrm{Gum}_i\}_{i \leqslant N}$, the i.i.d sequence of standard Gumbel random variables can be calculated by the following equation:

$$\mathrm{Gum}_i = -\log(-\log(U_i)), U_i \sim U(0,1). \quad (2)$$

Then the discrete random variable Ω will be transformed as follows:

$$\Omega^{'} = [\log \alpha_1 + \mathrm{Gum}_1, ... \log \alpha_i + \mathrm{Gum}_i], i \in [N] \quad (3)$$

This means each dimension of the original vector adds the corresponding Gum_i to get the new vector $\Omega^{'}$. To scale discrete random variables to achieve continuity, a softmax map indexed by a temperature parameter is introduced, therefore the concrete distribution $\sigma_\delta(\Omega_i^{'})$ can be defined as:

$$\sigma_\delta(\Omega_i^{'}) = e^{\frac{\Omega_i^{'}}{\delta}} / \sum_{i=1}^{N} e^{\frac{\Omega_i^{'}}{\delta}} \quad (4)$$

where δ represents the temperature parameter, $\Omega_i^{'}$ represents the i-th dimensional vector of $\Omega^{'}$.

4 METHODOLOGY

In this section, our proposed method SAGN is introduced in detail. Fig. 3 provides an overview of the SAGN framework. The algorithm is divided into two stages: offline training and online testing. In the offline training stage, feature extraction is performed on the historical data and the Gumbel-Softmax trick is used to initialize the correlation graph structure $\mathcal{G}_0$ for multi-stream $\mathcal{S}$. After this, the $\mathcal{G}_0$ is embedded into the diffusion convolutional gated recurrent unit networks with a sequence-to-sequence structure. During the GNN training, multiple well-performed GNNs are stored to initialize the GNN pool. In the online testing stage, we design a dynamic online adaptation technique to handle the concept drift problem. Specifically, the global correlation $\mathcal{G}_t$ of the multi-stream $\mathcal{S}$ is still preserved well and subgraphs are updated by new samples to adapt to drift. Besides, we design a dynamically updated validation set and a GNN pool rolling mechanism to reselect the optimal GNN periodically to ensure stable prediction performance.

4.1 Initialization (Offline Training)

In this section, we will give the details of offline training. We first initialize a sampled correlation graph structure $\mathcal{G}_0$ using Gumbel-Softmax trick for multi-stream $\mathcal{S}$. After that, $\mathcal{G}_0$ will be embedded into GNN training and a GNN pool will be initialized.

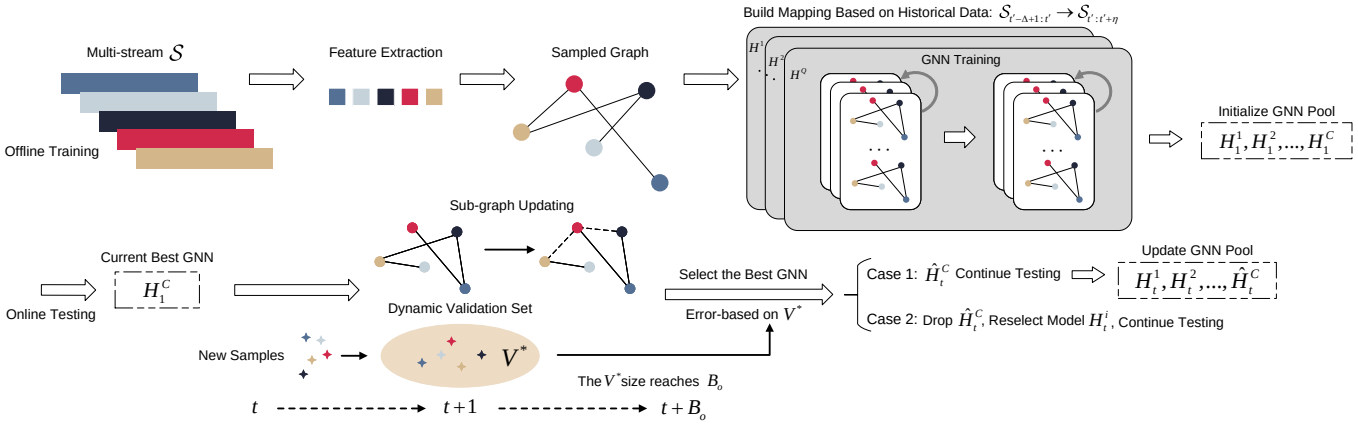


Fig. 3. Overview of the SAGN framework. SAGN is divided into two stages. The offline stage extracts the feature and initializes a sampled graph structure that contains correlation in multi-stream $\mathcal{S}$. The online stage use arriving new samples to update sub-graphs to adapt to unknown concept drift. The dynamic GNN pool rolling mechanism is designed for reselecting the best GNN in a certain cycle, and GNNs stored are revalidated periodically on the proposed dynamic validation set over time.

4.1.1 Initialization of the Correlation Graph Structure

Most existing GNN-based methods use pre-defined graphs (usually containing the location information of sensors) of the data set as the basis for constructing the graph topology. Considering that pre-graphs of real-world data sets may not be available, we construct the correlation graph structure based on historical data of multi-stream as the initialization.

First, We use the feature extraction network F_{etc} on multi-stream $\mathcal{S}$ to extract historical feature structure $F_{etc}(\mathcal{S})$ and reduce dimension. The feature extraction network F_{etc} consists of two convolutional layers, two normalization layers, and a fully connected layer [38].

After the calculation of F_{etc}, the data streams are combined in vector pairs, and the strength of the correlation between two streams is defined by probability $\mu_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle} \in [0, 1]$. The pairwise combination of vectors is obtained by concatenation and computation of two fully connected layers. The Gumbel-Soft trick is applied here to calculate the sampled graph structure on every two streams u and v: $G_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle}$, which is based on the discrete probability variable $\mu_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle}$ (see Eq. (2), (3) and (4)):

$$\mathcal{G}_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle} = softmax(\frac{\log(\frac{\mu_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle}}{1-\mu_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle}}) + (\mathrm{Gum}_u - \mathrm{Gum}_v)}{\delta}) \tag{5}$$

Let $\mathcal{G}_0 = [\mathcal{G}_{uv}]_{s \times s}, u, v \in [s]$ denoted as initialized correlation graph structure, where $\mathcal{G}_{uv} = G_{\langle \mathcal{D}^u, \mathcal{D}^v \rangle}$ (corresponding to Step 3 in Algorithm 1). It can be seen as the initialization correlation graph structure of multi-stream $\mathcal{S}$ and would be embedded in the GNN training process.

4.1.2 Initialization of the GNN Pool

The sequence-to-sequence model [42], which has an encoder-decoder structure is applied to build the mapping relationship between $\mathcal{S}_{t-\Delta+1,t}$ and $\mathcal{S}_{t+1:t+\eta}$ during GNN training process. The initialization correlation graph structure $\mathcal{G}_0$ is embedded in the recurrent convolutional framework to handle all streams simultaneously.

The hidden state of the sequence-to-sequence model updates from $\Theta_{t^*-1} \rightarrow \Theta_{t^*}$ at each time step t^*. The values of $\mathcal{S}_{t^*}$ in the defined window size Δ are the input. Specifically, the encoder part performs updates recurrently: $t^{'} - \Delta + 1 \rightarrow t^{'}$ and ends with Θ_{t^*} as the representation of $\mathcal{S}_{t^*}$. Correspondingly, the Θ_{t^*} is used for the initialization of the decoder. The hidden state performs the recurrent updates of each next η time steps in the future: $t^{'}+1 \rightarrow t^{'}+\eta$.

We use the Diffusion Convolutional Gated Recurrent Unit defined in [32] and the concatenation method for the graph structure and hidden statements in [38] to collaboratively complete GNN training process (corresponding to Steps 4 to 7 in Algorithm 1):

$$\begin{cases} r_{t^*} = sigmoid(W_r \star_{\mathcal{G}_0} [\mathcal{S}_{t^*} \parallel \Theta_{t^*-1}] + f_r), \\ q_{t^*} = \tanh(W_q \star_{\mathcal{G}_0} [\mathcal{S}_{t^*} \parallel (r_{t^*} \odot \Theta_{t^*-1}] + f_q), \\ c_{t^*} = sigmoid(W_c \star_{\mathcal{G}_0} [\mathcal{S}_{t^*} \parallel \Theta_{t^*-1}] + f_c), \\ \Theta_{t^*} = c_{t^*} \odot \Theta_{t^*-1} + (1 - c_{t^*}) \odot q_{t^*} \end{cases}$$

where $\star\mathcal{G}_0$ represents the graph diffusion convolution and can be defined as:

$$A \star_{\mathcal{G}_0} W_\lambda = \sum_{k=0}^{K} (w_{k,1}^{\lambda} (M_O^{-1} \mathcal{G}_0)^k + w_{k,2}^{\lambda} (M_I^{-1} \mathcal{G}_0{}^T)^k) A$$

where r_{t^*} and q_{t^*} are the reset gate and update gate at time step t^*, and M_O and M_I are the out-degree and in-degree matrix. W_λ for $\lambda = r, q, c$ are the model parameters for the corresponding filters, and diffusion degree k is a hyperparameter. Mean absolute error (MAE) between the η prediction values and ground truth per Δ is used as the loss function of the training process.

The GNN training will iterate many times according to pre-setting, and the latest well-performed GNNs are stored to initialize the GNN pool: $P = \{H_1^i(\cdot; \mathcal{G}_0^i, \Theta_0^i) | i \in [C]\}$. $\mathcal{G}_0^i$ are updated from the GNN training process (corresponding to Step 8 in Algorithm 1). In this paper, we store 20 GNNs in P. The scale of the GNN pool can be flexibly adjusted according to the actual hardware environment and the update speed of the multi-stream.

During the calculation of the graph convolution, both correlations and spatiotemporal dependencies of different streams are both taken into account. In addition, compared

to only using the best GNN, the GNN pool can preserve richer historical data of multi-stream.

4.2 Dynamic Online Self-Adaptation (Online Testing)

In this section, we will give the details of online testing. The dynamically updated validation set and GNN pool rolling mechanism are proposed to work in tandem to achieve the dynamic self-adaptation process. Globally the correlation structure is maintained well and sub-graphs will be updated by new arriving samples locally to adapt to concept drift.

4.2.1 Dynamically Updated Validation Set

The GNNs stored in P represent the historical data information of multi-stream S. But these GNNs cannot adapt to concept drift in future time steps directly. So we design the dynamic self-adaptation in the online testing stage, the adaptation process is performed by updating sub-graphs using new samples. To maintain the prediction performance of the framework, we will reselect the Best GNN from P periodically.

Therefore revalidating the performance of GNNs is necessary. In SAGN, we design *a dynamically updated validation set* V^* with size B_o on the test set to achieve it. V^* is updated with new samples over time (corresponding to Step 9 in Algorithm 1). Specifically, Once the count of arriving new samples achieves B_o, the validation process will start. Multiple GNNs in P and the current updated GNN will be validated on the current V^* (corresponding to Step 16 in Algorithm 1). And the best GNN will be used to process the prediction task for coming samples in the next time steps. The setting of B_o depends on the data set characteristics. For example, if the data set has a slow change cycle, we can set it to a larger size. When it is used for a multi-stream that may change rapidly, we appropriately reduce its size to ensure that the update cycle is maintained within the optimal learning period for the new data distribution.

The proposed dynamically updated validation set can ensure that the current GNN matches the latest data distribution, and the SAGN is able to maintain high-precision prediction performance for a long time.

4.2.2 Dynamic GNN Pool Rolling Mechanism

The updates for sub-graphs in SAGN are designed to adapt to concept drift, but sometimes erroneous updates are inevitable. To solve this problem, we design *a GNN pool rolling mechanism* to avoid this problem.

Specifically, suppose H_1^C is the best GNN in P, the online testing will start with H_1^C to predict the values of the next η time steps on the test set continuously. When the ground truth values of new samples are obtained, we use these values to update the sub-graphs of $\mathcal{G}_{t-1}^C$ to adapt to the unknown concept drift, and the GNN is also updated: $H_t^C \rightarrow \hat{H}_t^C, \mathcal{G}_{t-1}^C \rightarrow \mathcal{G}_t^C$ (corresponding to Step 11 to 14 in Algorithm 1). Therefore the GNN can learn and adapt to the drift. When the count of new sample arrival reaches B_o, the first dynamic validation set ${V_1}^*$ is generated automatically based on these new samples. Then, the loss values of all GNNs in P and $\hat{H}_t^C$ are validated on ${V_1}^*$. If $\hat{H}_t^C$ still reach the best performance on ${V_1}^*$, $\hat{H}_t^C$ replaces H_t^C, and P is updated. If not, for example, H_t^1 performs best on ${V_1}^*$, the current updated GNN $\hat{H}_t^C$ will be dropped. P would stay and H_t^1 is the prediction model in the next stage (corresponding to Step 16 in Algorithm 1). The rolling period of the mechanism depends on the size of B_o.

A stable prediction performance is guaranteed in this dynamic model selection and validation mechanism. Also, this enables SAGN to avoid performance degradation caused by erroneous updates.

4.2.3 The SAGN Algorithm

The pseudo-code of SAGN is shown in Algorithm 1. Steps 1 to 8 are about the offline training stage. First, we set several pre-settings for initialization. Then, the training process is performed after splitting the data set. Step 3 initializes the correlation graph structure $\mathcal{G}_0$. Steps 4 to 7 show the GNN iterative training process. The GNN pool is initialized with multiple well-performed GNNs as alternative models to the prediction task in the online self-adaptation stage in Step 8. Meanwhile, $\mathcal{G}_0$ is updated to $\mathcal{G}_t^i$ for these GNNs.

The dynamic self-adaptation process starts from Step 9. The prediction task is started from H_t^C and the updating for sub-graphs is performed simultaneously in Steps 10 to 14. When every dynamic validation set is generated, current GNNs in the pool will be revalidated and reselected again in Steps 15 to 17. This dynamic GNN pool rolling mechanism runs through the whole online testing stage. The overview of SAGN framework can be seen in the Fig. 3.

5 Experiments

In this section, we performed extensive experiments to demonstrate that the proposed SAGN outperforms existing prediction frameworks, including traditional methods, the adaptation method, and GNN-based methods in most cases. Furthermore, we construct synthetic data sets to demonstrate that SAGN can learn and adapt when severe drift occurs in multi-stream.

We first give details of three real-world data sets. The experiment settings, comparative baselines, and experiment results are also given in the corresponding subsection.

5.1 Data Sets

Three real-world large-scale benchmark data sets are used for our experiments, and the summary information of data sets is shown in Table 2.

TABLE 2
Summary Statistics of Data sets

Data set	Nodes	Samples	Sample Rate
METR-LA	207	34272	5min
PEMS-BAY	325	52116	5min
WEATHER ($psur$)	10	29639	1h
WEATHER ($q2m$)	10	29639	1h

(1)**METR-LA**: This is a traffic information data set that was collected from the loop detection sensors in highways in Los Angeles Country [44]. Four months of speed data streams(Mar 31_{st} 2012 - Jun 30_{st} 2012) generated by 207 sensors.

Algorithm 1 SAGN

1: Input Multi-stream $\mathcal{S}$, the window size of historical sequence for all streams Δ, the prediction time steps for coming samples η, training learning rate LR and epoch Q_{max}, online self-adaptation learning rate lr and epoch $Q^{'}_{max}$, the temperature for Gumbel sampling δ, loss function (MAE is selected) and optimization algorithm (Adam algorithm [43] is selected);
2: Split we divide the whole multi-stream data set into training $\mathcal{S}^{tr}$, validation $\mathcal{S}^{val}$ and test $\mathcal{S}^{te}$ sets along the timeline, B_t is the batch size for $\mathcal{S}^{tr}$, B_o is the size of the dynamic validation set V^*;
3: Initial Θ_0, $\mathcal{G}_0$ (see Eq.(5), $P = \emptyset$
for $Q = 1, 2, \ldots, Q_{max}$ **do** // Offline training based on historical data
  4: Fetch training data from $\mathcal{S}^{tr}$ with batch size B_t;
  5: Calculate $L = loss(H^Q\left(\mathcal{S}^{tr}_{t'-\Delta+1:t'}; \mathcal{G}_0, \Theta_0\right), \mathcal{S}^{tr}_{t'+1:t'+\eta})$;
  6: Update $\mathcal{G}_0, \Theta_0 = \text{Adam}(L)$;
  7: Calculate the loss on $\mathcal{S}^{val}$: $L_Q = loss(H^Q\left(\mathcal{S}^{val}_{t'-\Delta+1:t'}; \mathcal{G}_0, \Theta_0\right), S^{val}_{t'+1:t'+\eta})$ and save the model H^Q;
end
8: Initialize GNN Pool
select C well-performed GNNs from $\{H^1, H^2, \cdots, H^{Q_{max}}\}$ and initialize the GNN pool: $P = \{H^i_1(\cdot; \mathcal{G}^i_0, \Theta^i_0) | i \in [C]\}$. Suppose H^C_1 is the best one;
for *new samples in* $\mathcal{S}_{te}$ *arrive in each time step,* $t = 1, 2, 3, ...$ **do** // Online testing and adaptation
  9: Generate Dynamic Validation Set new samples are stored in V^* temporarily and its size up to B_o;
  10: Online Prediction $H^C_t\left(\mathcal{S}^{te}_{t-\Delta+1:t}; \mathcal{G}^C_{t-1}, \Theta^C_{t-1}\right) = S^{pre}_{t+1:t+\eta}$; // Give online prediction results
  if $t > 1$ **then**
    11: Obtain ground truth: $S^{true}_{t:t+\eta-1}$;
    for $Q^{'} = 1, 2, \ldots Q^{'}_{max}$ **do** // Sub-graphs updating
      12: Calculate $L = loss(\hat{H}^C_t\left(S^{te}_{t-\Delta:t-1}; \mathcal{G}^C_{t-1}, \Theta^C_{t-1}\right), S^{true}_{t:t+\eta-1})$;
      13: Update $\mathcal{G}^C_{t-1}, \Theta^C_{t-1} = \text{Adam}(L)$, H^C_t;
    end
  end
  14: Update $\mathcal{G}^C_t = \mathcal{G}^C_{t-1}$, $\hat{H}^C_t = H^C_t$;
end
if *the size of* V^* *reaches* B_o **then**
  15: Calculate $L_i = loss(H^i_t\left(V^*_{t-\Delta+1:t}; \mathcal{G}^i_{t-1}, \Theta^i_{t-1}\right), V^*_{t+1:t+\eta})$, $i \in [C]$; $\hat{L}_C = loss(\hat{H}^C_t\left(V^*_{t-\Delta+1:t}; \mathcal{G}^C_{t-1}, \Theta^C_{t-1}\right), V^*_{t+1:t+\eta})$
  16: Validate and reselect the GNN
  if $\hat{L_C} < \arg\min_{i \in [C]}\{L_1, L_2, ..L_C\}$ **then** // Dynamic model selection
    $\hat{H}^C_t$ is selected for the prediction GNN in the next stage;
    $H^C_t = \hat{H}^C_t$, update the GNN pool P;
  else
    drop: $\hat{H}^C_t$;
    select he model H^i_t corresponding to $\arg\min_{i \in [C]}\{L_1, L_2, ..L_C\}$ as the prediction GNN for the next stage;
  end
  17: reset V^*
end

(2)**PEMS-BAY**: This is a traffic data set collected by the California Transportation Agencies (CalTrans) Performance Measurement System [45]. Six months of speed data streams (Jan 1_{st} 2017 - May 31_{st} 2017) generated by 325 sensors.

(3)**WEATHER FORECASTING**: This is a publicly available dataset compiled from weather stations in Beijing, China [46]. In the remainder of this paper, it is abbreviated as WEATHER. This data set comprises hourly weather data from ten weather stations (from Jan 1_{st} 2015 - May 31_{st} 2018, which is 1188 days in total). It comprises weather monitoring features from $3:00$ intraday to $15:00$ the next day. Each weather station has the same weather monitoring features. We selected two weather features $psur$ and $q2m$ for the experiments. They represent the surface air pressure (unit ℃) and the specific humidity at a height of two meters above the ground (unit *g/kg*), respectively.

The same data pre-processing procedure as in [32] is performed for METR-LA and PEMS-BAY data sets. For WEATHER, the data preprocessing is performed by removing the overlapping parts on the timeline, and the missing values are replaced by the mean of the preceding and following time steps. We apply Z-Score normalization to all inputs. For METR-LA and PEMS-BAY, 70% of the data is used for training, 10% is used for validation, and the remaining 20% is used for testing. For WEATHER, 40% of the data is used for training, 10% is used for validation, and the remaining 50% is used for testing. The training and validation batch size is 64 for all data sets, and we simulate the online multi-stream environment in the form of samples arriving sequentially at a single time step.

5.2 Experiment Settings

5.2.1 Baseline Methods

Since the current research on multi-stream concept drift is still not well solved in the field, the aim of this paper is to develop a solution to handle it. Some existing GNN-based methods are chosen for comparison because our study reconsiders the learning procedure of GNN-based predictors from an aspect of concept drift adaptation for multi-stream. In addition, some traditional models based on statistics and the adaptation method designed for multi-stream are considered accordingly:

- Traditional non-deep learning methods:
  (1)**HA**: Historical Average, which achieves the prediction task using the weighted average of historical data;
  (2)**ARIMA**$_{kal}$: Auto-Regressive Integrated Moving Average model with the Kalman filter;
  (3)**VAR**: Vector Auto-Regression;
  (4)**SVR**: Support Vector Regression, which uses a linear support vector to achieve the regression.
- Traditional deep learning methods:
  (1)**FNN**: Feedforward Neural Network, which includes two hidden layers and $L2$ regularization;
  (2)**FC-LSTM**: Fully Connected LSTM, which means the recurrent neural network is connected to the LSTM hidden units.
- Multi-stream adaptation method:
  (1)**MuNet** [10]: A Bayesian neural network-based multi-stream adaptation framework. Due to the limitation of the connector on the expansion performance, the algorithm can only be effective for the concurrent drift problem in multi-stream with a small data scale. For the fairness of comparison, we put SAGN and MuNet in the same experiment setting to compare. Specifically, we choose the same ten streams from METR-LA and PEMS-BAY data sets separately for these two adaptation methods (we use SAGN (10) and MuNet (10) to represent them).
- GNN-based methods:
  (1)**Graph-WaveNet** [33]: Graph WaveNet learns an adaptive dependency matrix through node embedding, combining graph convolution and spatiotemporal convolution to capture the temporal and spatial dependencies ;
  (2)**DGCRN** [35]: Dynamic Graph Convolutional Recurrent Network integrates dynamic graphs generated by dynamic filters at each time step and predefined graphs into a dynamic convolutional recurrent network to perform the prediction task;
  (3)**ASTGCN** [36]: Attention-based Spatial-Temporal Graph Convolution Networks design the spatial-temporal attention mechanism to synergistic graph convolution for weighted fusion of temporal and spatial features;
  (4)**AGCRN** [37]: Adaptive Graph Convolutional Recurrent Network designs two adaptive blocks of node adaptive learning and data adaptive graph to automatically infer the dependencies between streams;
  (5)**DCRNN** [32]: Diffusion Convolutional Recurrent Neural Network are designed for performing the graph convolution;
  (6)**GTS** [38]: Graph for Time Series designs the discrete graph learning structure to capture the dependencies between streams.

5.2.2 Settings

Platform. We use PyTorch 1.7.1 to implement the model. All experiments are run on a server with the Nvidia Quadro RTX 8000 GPU (48GB). The CPU is Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz, and the total available memory of the server is 187GB.

Parameter Settings. The temperature parameter δ of the Gumbel-Softmax trick is set to 0.9. The Adam optimizer is selected to train the model. The initial learning rate is set to 0.005 for METR-LA, WEATHER and all synthetic data sets, and 0.001 for PEMS-BAY. The training epoch is set to 200. The online learning rate e for dynamic online self-adaptation is set to 0.00001 for METR-LA, and 0.000005 for PEMS-BAY and WEATHER. The size of the dynamic validation set (B_o) is set to 144 for METR-LA, PEMS-BAY, and 24 for WEATHER. The online learning epoch number is set to 3 for all data sets. Since the dataset WEATHER does not have a pre-defined graph, therefore when comparing the methods using pre-defined graphs, we use the nearest neighbor matrix calculated by $neighbors_graph$ in Python package $sklearn$ instead of pre-defined graphs. The k is set to 5. The size of the GNN pool is set to 20 for all data sets.
Evaluation Metrics. The predicted values are transformed back to the actual values first and we compare them with the ground truth. Mean absolute error (MAE), mean percentage absolute error (MAPE) and root mean square error (RMSE) are used for the evaluation metrics for all experiments. To ensure the fairness of the comparison, the results on all data sets are the average of five repeated times.

5.3 Result Analysis

5.3.1 Overall Performance

Table 3 compares the performance of SAGN against all baseline methods on three real-world data sets. SAGN achieves state-of-art prediction performance in most cases. On the more commonly used traffic prediction datasets, the improvement is about 7% on METR-LA, and about 3% on PEMS-BAY. Especially in multi-step prediction, SAGN has obvious advantages. Since the vehicle speed recorded by the two traffic data sets shows a periodic change from day to night, learning historical data is useful for the prediction task. However, unexpected accidents on certain roads will cause drift to varying degrees and ranges, and this drift information will not be able to be learned from historical data. We also give the prediction results of the two weather indicators ($pusr$ and $q2m$) in the WEATHER data set. Compared to the above two traffic data sets, the weather data presents a more regular change trend and fluctuation range. This is the reason why some traditional regression prediction methods are still applicable. But despite this, SAGN's performance on the prediction tasks on the two weather indicators still exceeds most methods. For comparison with the multi-steam adaptation method MuNet, SAGN can also achieve great performance even on a small data scale.

5.3.2 Ablation Study

We construct synthetic data sets to demonstrate the self-adaptation effectiveness of SAGN, especially for sudden drifts that diverge significantly from the original distribution [47]. We design two fundamental multi-stream data sets, $\mathcal{S}^a$ and $\mathcal{S}^b$. In $\mathcal{S}^a$, there is no correlation between streams, and there is a correlation between streams in $\mathcal{S}^b$. Based on this setting, we add different types of functions to simulate drift at a certain time step $\hat{t}$ on the two multi-streams, and the drift-inducing functions cover multiple function types.

TABLE 3
The performance comparison between SAGN and the baseline methods on real-world data sets

METR-LA	Horizon 3			Horizon 6			Horizon 12		
Methods | Metrics	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	4.16	7.80	13.00%	4.16	7.80	13.00%	4.16	7.80	13.00%
ARIMA	3.99	8.21	9.60%	5.15	10.45	12.70%	6.90	13.23	17.40%
VAR	4.42	7.89	10.20%	5.41	9.13	12.70%	6.52	10.11	15.80%
SVR	3.99	8.45	9.30%	5.05	10.87	12.10%	6.72	13.76	16.70%
FNN	3.99	7.94	9.90%	4.23	8.17	12.90%	4.49	8.69	14.00%
FC-LSTM	3.44	6.30	9.60%	3.77	7.23	10.90%	4.37	8.69	13.20%
Graph-WaveNet	2.69	5.15	6.90%	3.07	6.22	8.37%	3.53	7.37	10.10%
DGCRN	2.62	5.01	6.63%	2.99	6.05	8.02%	3.44	7.19	9.73%
ASTGCN	4.86	9.27	9.21%	5.43	10.61	10.13%	6.51	12.52	11.64%
AGCRN	2.87	5.58	7.70%	3.23	6.58	9.00%	3.62	7.51	10.38%
DCRNN	2.77	5.38	7.30%	3.15	6.45	8.80%	3.60	7.59	10.50%
GTS	2.64	4.95	6.80%	3.01	5.85	8.20%	3.41	6.74	9.90%
MuNet (10)	10.02	10.02	15.73%	10.06	10.06	15.79%	10.13	10.13	15.97%
SAGN (10)	2.56	3.98	6.60%	2.95	4.71	7.92%	3.50	5.67	9.69%
SAGN	**2.46**	**4.41**	**6.41%**	**2.79**	**5.15**	**7.64%**	**3.16**	**5.91**	**9.10%**

PEMS-BAY	Horizon 3			Horizon 6			Horizon 12		
Methods | Metrics	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	2.88	5.59	6.80%	2.88	5.59	6.80%	2.88	5.59	6.80%
ARIMA	1.62	3.30	3.50%	2.33	4.76	5.40%	3.38	6.50	8.30%
VAR	1.74	3.16	3.60%	2.32	4.25	5.00%	2.93	5.44	6.50%
SVR	1.85	3.59	3.80%	2.48	5.18	5.50%	3.28	7.08	8.00%
FNN	2.20	4.42	5.19%	2.30	4.63	5.43%	2.46	4.98	5.89%
FC-LSTM	2.05	4.19	4.80%	2.20	4.55	5.20%	2.37	4.96	5.70%
Graph-WaveNet	1.30	2.74	**2.73%**	1.63	3.70	3.67%	1.95	4.52	4.63%
DGCRN	**1.28**	2.69	2.66%	**1.59**	3.63	**3.55%**	1.89	4.42	4.43%
ASTGCN	1.52	3.13	3.22%	2.01	4.27	4.48%	2.61	5.42	6.00%
AGCRN	1.37	2.87	2.94%	1.69	3.85	3.87%	1.96	4.54	4.64%
DCRNN	1.38	2.95	2.90%	1.74	3.97	3.90%	2.07	4.74	4.90%
GTS	1.32	2.62	2.80%	1.64	3.41	3.60%	1.91	3.97	4.40%
MuNet (10)	5.26	5.26	18.36%	5.31	5.31	18.46%	5.46	5.46	18.71%
SAGN (10)	1.58	2.43	3.77%	2.08	3.30	5.36%	2.55	4.06	6.93%
SAGN	**1.30**	**2.43**	**2.75%**	**1.60**	**3.11**	**3.61%**	**1.85**	**3.61**	**4.37%**

WEATHER ($psur$)	Horizon 3			Horizon 6			Horizon 12		
Methods | Metrics	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	28.52	39.96	3.00%	28.52	39.96	3.00%	28.52	39.96	3.00%
ARIMA	2.16	2.16	0.22%	2.91	2.91	0.29%	4.30	4.30	0.43%
VAR	1.56	4.60	0.16%	2.30	5.36	0.23%	3.09	6.02	0.31%
SVR	1.62	5.21	0.16%	2.57	6.29	0.26%	3.58	7.21	0.36%
FNN	5.16	7.27	0.53%	5.94	8.10	0.60%	6.98	9.15	0.70%
FC-LSTM	2.06	5.00	0.21%	2.86	5.80	0.29%	3.71	6.54	0.38%
Graph-WaveNet	1.00	4.80	**0.10%**	1.44	5.13	**0.15%**	2.25	5.57	**0.23%**
DGCRN	**0.94**	4.33	**0.10%**	1.56	5.09	0.16%	2.71	6.11	0.28%
ASTGCN	1.52	4.50	0.14%	2.15	5.32	0.22%	2.90	5.94	0.30%
AGCRN	1.03	4.77	0.11%	1.48	5.11	**0.15%**	2.31	5.60	0.24%
DCRNN	1.06	4.81	0.11%	1.51	5.23	0.16%	2.37	5.24	0.24%
GTS	1.04	2.86	0.11%	1.55	3.39	0.16%	2.49	4.30	0.25%
MuNet	3.42	3.42	0.35%	3.45	3.45	0.35%	3.58	3.58	0.36%
SAGN	**0.96**	**1.29**	**0.10%**	**1.43**	**1.79**	**0.15%**	**2.29**	**2.67**	**0.23%**

WEATHER ($q2m$)	Horizon 3			Horizon 6			Horizon 12		
Methods | Metrics	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	5.15	6.29	191.12%	5.15	6.29	191.12%	5.15	6.29	191.12%
ARIMA	**0.13**	**0.13**	12.66%	**0.20**	**0.20**	20.34%	**0.25**	**0.25**	24.09%
VAR	0.61	1.04	12.64%	0.86	1.34	18.84%	1.17	1.72	27.73%
SVR	0.25	0.58	13.32%	0.38	0.74	21.32%	0.53	0.89	32.70%
FNN	0.85	1.01	65.14%	1.00	1.18	79.11%	1.19	1.37	95.90%
FC-LSTM	0.30	0.58	19.28%	0.43	0.73	29.07%	0.59	0.88	42.29%
Graph-WaveNet	0.56	1.00	**10.38%**	0.79	1.30	**15.30%**	1.10	1.70	**23.33%**
DGCRN	0.56	1.01	10.89%	0.81	1.34	16.61%	1.17	1.79	25.83%
ASTGCN	0.61	1.02	13.39%	0.86	1.34	17.63%	1.16	1.72	25.11%
AGCRN	0.59	1.03	11.20%	0.82	1.33	16.13%	1.16	1.74	25.00%
DCRNN	0.59	1.03	11.20%	0.82	1.34	16.45%	1.16	1.77	24.98%
GTS	0.60	0.88	11.26%	0.84	1.17	16.47%	1.16	1.53	24.43%
MuNet	1.33	1.33	42.04%	1.35	1.35	42.55%	1.55	1.55	46.65%
SAGN	**0.56**	**0.69**	**10.72%**	**0.79**	**0.93**	**16.02%**	**1.10**	**1.25**	**24.24%**

The first-class multi-stream $\mathcal{S}^a$ is initialized by the function $RegressionGenerator$ in the Python package $scikit-multiflow$. There are ten streams in $\mathcal{S}^a$ and the sample number of all streams is $30,000$. Ten streams have different parameter settings of $ramdom_state$ (from 0 to 9). To further distinguish the data streams, we add different degrees of offsets to all streams on this basis, with offsets of $20, 40, 70, 60, 15, 45, 57, 82, 26, 33$. These offsets are set randomly and have no special meaning. The second-class multi-stream $\mathcal{S}^b$ is derived from the real-world data set PEMS-BAY. Specifically, the data streams in PEMS-BAY are related to each other, so we transform on the part of PEMS-BAY to construct $\mathcal{S}^b$. We select the first $30,000$ samples of ten streams in PEMS-BAY as the initialization for $\mathcal{S}^b$.

The sudden drift starts at the time step of the 15001_{st} sample in all streams of both $\mathcal{S}^a$ and $\mathcal{S}^b$. Multiple drift function types are considered to ensure completeness of the verification process, including linear, trigonometric, and polynomial functions. The details of these functions are defined as follows:

$$\begin{cases} \mathcal{S}^*_{lir} = 5\mathcal{S}^* + 50 \\ \mathcal{S}^*_{tri} = \sin(5\mathcal{S}^*) + 50 \\ \mathcal{S}^*_{pol} = 0.01(\mathcal{S}^*)^{2.5} - 0.1(\mathcal{S}^*)^2 + 0.5\mathcal{S}^* \end{cases} \tag{6}$$

Each time step is separated by one hour for all synthetic data sets. Therefore, we generate a total of eight synthetic multi-stream data sets according to the above settings. They are $\mathcal{S}^a_{ndf}$, $\mathcal{S}^a_{lir}$, $\mathcal{S}^a_{tri}$, $\mathcal{S}^a_{pol}$, and $\mathcal{S}^b_{ndf}$, $\mathcal{S}^b_{lir}$, $\mathcal{S}^b_{tri}$, $\mathcal{S}^b_{pol}$. The subscript ndf represents that no drift function is added in the multi-stream. lir, tri, and pol represent the different drift functions (as shown in Eq. (6)).

The ablation study experiment settings are as follows. For all synthetic data sets, We use 40% for training, 10% for validation, and the last 50% for testing. The learning rate of the dynamic online self-adaptation is 0.005 for $\mathcal{S}^a_{lir}$, 0.0005 for $\mathcal{S}^a_{tri}$, $\mathcal{S}^a_{pol}$, $\mathcal{S}^b_{lir}$, $\mathcal{S}^b_{pol}$, 0.00005 for $\mathcal{S}^b_{ndf}$, $\mathcal{S}^b_{tri}$, 0.000005 for $\mathcal{S}^a_{ndf}$. The size of the dynamic validation set is 144 for $\mathcal{S}^b_{ndf}$ and 24 for the others.

The experiment results are shown in Table 4. We select the GTS method as the baseline to compare in this part because it is the best-performance GNN model without using any pre-defined graph information. It can be seen that the GTS maintains good prediction performance when the multi-stream does not drift. But when the data distribution changes drastically after a certain time step, it is difficult for the GTS to adapt. Judging from the prediction results, GTS suffers severe performance slippage in multi-stream with sudden drift, while SAGN is not affected very much. Precisely, after building a correlation graph structure globally, SAGN maintains good prediction performance by continuously updating sub-graphs with arriving new samples. SAGN is able to learn the new data distribution after concept drift through a dynamic online adaptation process.

In practical situations, the adaptation to sudden drift is particularly important, especially for the pre-judgment of some important detection indicators. Rapid adaptation means that it can quickly help decision-making and avoid major losses. To show the adaptation performance of the proposed SAGN more concretely, we draw the figure of the MAE change when concept drift occurs on multi-stream.

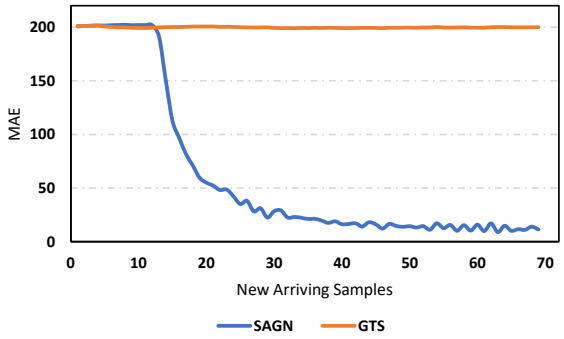


Fig. 4. The adaptation performance comparison

The comparison experiment is tested on the synthetic data set $\mathcal{S}^a_{lir}$. The results are shown in Fig. 4, SAGN can quickly adapt to the new data distribution after about ten new samples arrive, while GTS cannot.

We also tested the impact of the temperature parameter δ in Gumbel-Softmax trick. We report the performance on the METR-LA data set. This parameter determines the smoothness of sampling results. The smaller this parameter, the more sampling results will tend to one-hot encoding, but the gradient changes greatly. The higher this parameter, the smoother the sampling result, but the variance of the gradient is small [40]. Fig. 5 shows the effect of this parameter.

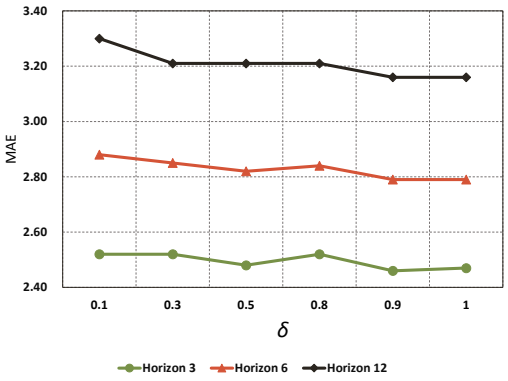


Fig. 5. The effect of the δ in Gumbel-Softmax trick

The main difference between SAGN and other GNN-based methods is that our testing stage runs online. The concept drift self-adaptation progress is achieved by sub-graphs updating when new samples arrive. We compare the run time (as shown in Table 5) with GTS in the training and testing stages separately. The experiment is performed on the METR-LA data set. In the training stage, our average training time per epoch is slightly shorter than GTS. In the test stage, neither GTS nor other traditional GNN-based methods consider concept drift adaptation, and the prediction modes are all performed on batch data in offline mode. But SAGN predicts the incoming new samples one by one according to the data stream online mode, and the average prediction time is $0.34s$, which is also much smaller than a single time step.

TABLE 4
The performance comparison for SAGN and GTS under sudden drift

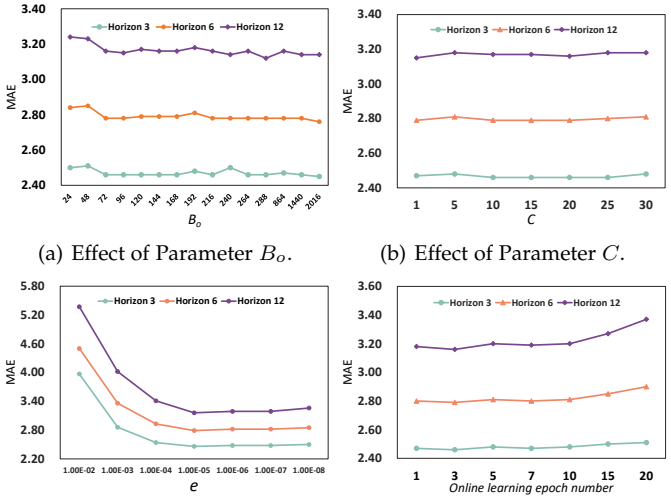
		Horizon 3			Horizon 6			Horizon 12		
Data Sets	Methods	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
$\mathcal{S}^a_{ndf}$	GTS	0.80	1.00	**2.34%**	0.80	1.00	**2.34%**	0.80	1.00	**2.34%**
	SAGN	**0.80**	**0.97**	2.35%	**0.80**	**0.97**	2.35%	**0.80**	**0.97**	2.35%
$\mathcal{S}^a_{lir}$	GTS	199.86	218.02	81.97%	199.23	217.35	81.70%	199.31	217.11	81.92%
	SAGN	**5.40**	**6.65**	**2.73%**	**5.40**	**6.64**	**2.73%**	**5.43**	**6.69**	**2.75%**
$\mathcal{S}^a_{tri}$	GTS	5.47	6.98	10.92%	5.81	7.72	11.62%	8.90	10.98	17.40%
	SAGN	**0.67**	**0.76**	**1.34%**	**0.67**	**0.76**	**1.34%**	**0.67**	**0.76**	**1.35%**
$\mathcal{S}^a_{pol}$	GTS	27.70	28.09	155.04%	27.44	27.91	153.48%	27.23	27.73	152.37%
	SAGN	**0.62**	**0.74**	**3.47%**	**0.62**	**0.73**	**3.46%**	**0.62**	**0.74**	**3.49%**
$\mathcal{S}^b_{ndf}$	GTS	9.00	16.56	3.74%	12.79	24.05	5.71%	18.06	32.94	8.59%
	SAGN	**8.67**	**13.20**	**3.61%**	**12.11**	**18.83**	**5.43%**	**16.50**	**25.41**	**8.02%**
$\mathcal{S}^b_{lir}$	GTS	53.76	57.01	43.07%	50.88	56.07	42.65%	52.47	59.91	44.79%
	SAGN	**7.67**	**10.06**	**7.27%**	**9.11**	**12.48**	**9.02%**	**11.30**	**15.43**	**11.40%**
$\mathcal{S}^b_{tri}$	GTS	96.20	96.38	192.45%	144.09	144.26	288.22%	135.84	136.65	271.68%
	SAGN	**0.95**	**1.10**	**1.78%**	**0.98**	**1.14**	**1.84%**	**1.01**	**1.16**	**1.86%**
$\mathcal{S}^b_{pol}$	GTS	164.91	170.57	517.10%	215.50	222.99	668.66%	243.28	249.56	747.18%
	SAGN	**3.19**	**4.52**	**28.34%**	**3.67**	**5.37**	**39.07%**	**4.36**	**6.44**	**54.13%**

TABLE 5
The Runtime Comparison

Method	METR-LA	
	training time(s/epoch)	testing time(s/samples)
SAGN	48.87	0.34
GTS	49.24	0.0006

5.3.3 Parameter Sensitivity

We tested the parameter sensitivity of SAGN. We report MAE on the METR-LA data set. Whenever we change a test parameter, the other parameter values will keep their original default values.



(a) Effect of Parameter B_o.
(b) Effect of Parameter C.
(c) Effect of Parameter e.
(d) Effect of online learning epoch number.

Fig. 6. The effect of different parameters in the adaptation stage.

First, we tested parameter B_o, which is the size of the dynamic validation set. This size means the period of revalidation and reselection of the model. Generally speaking, for data streams that may change drastically, we can reduce this value to shorten the period of model revalidation. For data streams that may have periodic changes or a slow overall rate of change, we can appropriately increase this value to save computing resources. The dynamic validation set ensures that SAGN neither misses learning of new sample distributions nor overfitting. We tested this parameter range from 24 to 2016 (as shown in Fig. 6 (a)), and the corresponding update period of the dynamic validation set is two hours to one week. Due to the strong periodicity, it can be seen that changes to this value do not have a significant impact on performance for the data set METR-LA.

Then we tested parameter C, which means how many GNNs we save in the GNN pool. The GNN pool can save more historical information than only saving one GNN. We tested varied sizes of GNN pool, as shown in Fig. 6 (b) and based on the nature of the data set, the change of this parameter will not have a huge influence on the performance in the current data set.

The online learning rate e is also an important parameter in our method which means the degree of model learning after the new samples arrive. Since the correlation structure remains stable globally, the purpose of the self-adaptation process is to update sub-graphs changes locally caused by concept drift. Therefore, the e should not be set too large. If it is set too large, the model will be at risk of overfitting. Fig. 6 (c) shows its influence.

Another parameter is the number of the online learning epoch, which has a similar effect to parameter e (as shown in Fig. 6 (d)). It should be noted that we only need to adjust and update sub-graphs that contain correlation changes locally in our self-adaptation process. So this parameter should be set to a suitable value because if this parameter is set too large, the online prediction speed will be greatly reduced, which will go against the original intention of our online prediction method.

6 Conclusions and Further Study

In this paper, we propose a dynamic GNN-based self-adaptation method SAGN, which is designed for multi-stream under concept drift. Without using predefined graphs, we initialize a sampled correlation graph structure $\mathcal{G}_0$ based on historical data and it is embedded into GNN training. Multiple well-performed GNNs are stored in a GNN pool in the offline stage. When online testing starts, the dynamic adaptation process is performed by locally updating sub-graphs after new samples arrive. Meanwhile, globally the correlation graph structure of $\mathcal{S}$ will be preserved well. The designed dynamic GNN pool rolling mechanism cooperates with the dynamic validation set to periodically select the optimal GNN for the prediction task periodically. We tested the prediction performance of SAGN and baseline methods on real-world and synthetic multi-stream data sets. The experiment results show that our method performs the best in most cases.

In future work, we will consider more complicated drift situations in multi-stream from the multi-variate perspective. For example, due to hardware and environment limitations, multiple weather observation stations may not necessarily have the same number of sensors with identical performance, and the problem of no common feature time series among multiple streams will become a new challenge. Meanwhile, given some practical cases, for example, sometimes it is hard to get enough data in the beginning for training. We hope to use less historical data to achieve an efficient initialized model building. In addition, we will consider making the multi-stream self-adaptation framework more lightweight to flexibly adapt to the increasing online prediction needs.

Acknowledgments

The work presented in this paper was supported by the Australian Research Council (ARC) under Laureate project FL190100149 and discovery project DP200100700.

References

[1] B. Tran, A. D. Nguyen, L. N. Van, and K. Than, "Dynamic transformation of prior knowledge into bayesian models for data streams," *IEEE Transactions on Knowledge and Data Engineering*, 2021.

[2] Y. Gao, S. Chandra, Y. Li, L. Khan, and T. Bhavani, "Saccos: A semi-supervised framework for emerging class detection and concept drift adaption over data streams," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 3, pp. 1416–1426, 2022.

[3] M. Jain, G. Kaur, and V. Saxena, "A k-means clustering and svm based hybrid concept drift detection technique for network anomaly detection," *Expert Systems with Applications*, vol. 193, p. 116510, 2022.

[4] J. Lu, A. Liu, Y. Song, and G. Zhang, "Data-driven decision support under concept drift in streamed big data," *Complex & intelligent systems*, vol. 6, no. 1, pp. 157–163, 2020.

[5] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM computing surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.

[6] A. Liu, Y. Song, G. Zhang, and J. Lu, "Regional concept drift detection and density synchronized drift adaptation," in *IJCAI International Joint Conference on Artificial Intelligence*, Melbourne, Australia, August 19-25, 2017.

[7] W. Liu, H. Zhang, Z. Ding, Q. Liu, and C. Zhu, "A comprehensive active learning method for multiclass imbalanced data streams with concept drift," *Knowledge-Based Systems*, vol. 215, p. 106778, 2021.

[8] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 6, pp. 1532–1545, 2016.

[9] P. Castro-Cabrera, G. Castellanos-Dominguez, C. Mera, L. Franco-Marín, and M. Orozco-Alzate, "Adaptive classification using incremental learning for seismic-volcanic signals with concept drift," *Journal of Volcanology and Geothermal Research*, vol. 413, p. 107211, 2021.

[10] M. Zhou, Y. Song, G. Zhang, B. Zhang, and J. Lu, "An efficient bayesian neural network for multiple data streams," in *2021 International Joint Conference on Neural Networks (IJCNN)*. Shenzhen, China, July 18-22: IEEE, 2021, pp. 1–8.

[11] M. D. Jones, H. L. Peterson, J. J. Pierce, N. Herweg, A. Bernal, H. Lamberta Raney, and N. Zahariadis, "A river runs through it: A multiple streams meta-review," *Policy Studies Journal*, vol. 44, no. 1, pp. 13–36, 2016.

[12] A. Haque, H. Tao, S. Chandra, J. Liu, and L. Khan, "A framework for multistream regression with direct density ratio estimation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[13] L. Yang and A. Shami, "A lightweight concept drift detection and adaptation framework for iot data streams," *IEEE Internet of Things Magazine*, vol. 4, no. 2, pp. 96–101, 2021.

[14] Y. Song, G. Zhang, H. Lu, and J. Lu, "A fuzzy drift correlation matrix for multiple data stream regression," in *2020 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*. Glasgow, United Kingdom, July 19-24: IEEE, 2020, pp. 1–6.

[15] B. Dong, Y. Li, Y. Gao, A. Haque, L. Khan, and M. M. Masud, "Multistream regression with asynchronous concept drift detection," in *2017 IEEE International Conference on Big Data (Big Data)*. Boston, MA, USA, December 11-14: IEEE, 2017, pp. 596–605.

[16] S. Chandra, A. Haque, L. Khan, and C. Aggarwal, "An adaptive framework for multistream classification," in *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, Indianapolis, IN, USA, October 24-28, 2016, pp. 1181–1190.

[17] Y.-F. Li, Y. Gao, G. Ayoade, H. Tao, L. Khan, and B. Thuraisingham, "Multistream classification for cyber threat data with heterogeneous feature space," in *The World Wide Web Conference*. San Francisco, CA, USA, May 13-17: Association for Computing Machinery, 2019, p. 2992–2998.

[18] E. Yu, Y. Song, G. Zhang, and J. Lu, "Learn-to-adapt: Concept drift adaptation for hybrid multiple streams," *Neurocomputing*, vol. 496, pp. 121–130, 2022.

[19] S. Guo, Y. Lin, H. Wan, X. Li, and G. Cong, "Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting," *IEEE Transactions on Knowledge and Data Engineering*, 2021.

[20] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM international conference on data mining*. Minneapolis, Minnesota, USA, April 26-28: SIAM, 2007, pp. 443–448.

[21] F. Dong, G. Zhang, J. Lu, and K. Li, "Fuzzy competence model drift detection for data-driven decision support systems," *Knowledge-Based Systems*, vol. 143, pp. 284–294, 2018.

[22] Y. Song, J. Lu, H. Lu, and G. Zhang, "Fuzzy clustering-based adaptive regression for drifting data streams," *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 3, pp. 544–557, 2019.

[23] D. Liu, Y. Wu, and H. Jiang, "Fp-elm: An online sequential learning algorithm for dealing with concept drift," *Neurocomputing*, vol. 207, pp. 322–334, 2016.

[24] S. Xu and J. Wang, "Dynamic extreme learning machine for data stream classification," *Neurocomputing*, vol. 238, pp. 433–449, 2017.

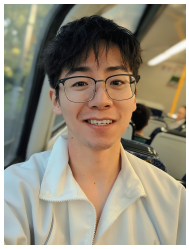
[25] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, "Extreme learning machine: theory and applications," *Neurocomputing*, vol. 70, no. 1-3, pp. 489–501, 2006.

[26] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*, Boston, MA, USA, August 20-23, 2000, pp. 71–80.

[27] G. Hulten, L. Spencer, and P. Domingos, "Mining time-changing data streams," in *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, San Francisco, California, USA, August 14-19, 2001, pp. 97–106.

[28] J. Gama, R. Rocha, and P. Medas, "Accurate decision trees for mining high-speed data streams," in *Proceedings of the ninth ACM*

SIGKDD international conference on Knowledge discovery and data mining, Washington, DC, USA, August 24-27, 2003, pp. 523–528.
[29] Y. Sun, K. Tang, Z. Zhu, and X. Yao, "Concept drift adaptation by exploiting historical knowledge," *IEEE transactions on neural networks and learning systems*, vol. 29, no. 10, pp. 4822–4832, 2018.
[30] D. Brzezinski and J. Stefanowski, "Reacting to different types of concept drift: The accuracy updated ensemble algorithm," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 25, no. 1, pp. 81–94, 2013.
[31] Y. Song, G. Zhang, H. Lu, and J. Lu, "A noise-tolerant fuzzy c-means based drift adaptation method for data stream regression," in *2019 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*. New Orleans, Louisiana, USA, June 23-26: IEEE, 2019, pp. 1–6.
[32] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *International Conference on Learning Representations (ICLR '18)*, Vancouver, BC, Canada, April 30-May 3, 2018.
[33] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, Vienna, Austria, May 03-07, 2019, pp. 1907–1913.
[34] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 01, New York, USA, February 7–12, 2020, pp. 1234–1241.
[35] F. Li, J. Feng, H. Yan, G. Jin, F. Yang, F. Sun, D. Jin, and Y. Li, "Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 2021.
[36] K. Guo, Y. Hu, Z. Qian, Y. Sun, J. Gao, and B. Yin, "Dynamic graph convolution network for traffic forecasting based on latent network of laplace matrix estimation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, pp. 1009–1018, 2022.
[37] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," *Advances in neural information processing systems*, vol. 33, pp. 17 804–17 815, 2020.
[38] C. Shang, J. Chen, and J. Bi, "Discrete graph structure learning for forecasting multiple time series," *International Conference on Learning Representations (ICLR '21)*, 2021.
[39] Y. Song, J. Lu, A. Liu, H. Lu, and G. Zhang, "A segment-based drift adaptation method for data streams," *IEEE transactions on neural networks and learning systems*, 2021.
[40] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *International Conference on Learning Representations (ICLR '17)*, Palais des Congrès Neptune, Toulon, France, April 24-26, 2017.
[41] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *International Conference on Learning Representations (ICLR '17)*, Palais des Congrès Neptune, Toulon, France, April 24-26, 2017.
[42] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27, Palais des Congrès de Montréal, Montréal CANADA, December 8-13, 2014.
[43] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
[44] H. V. Jagadish, J. Gehrke, A. Labrinidis, Y. Papakonstantinou, J. M. Patel, R. Ramakrishnan, and C. Shahabi, "Big data and its technical challenges," *Communications of the ACM*, vol. 57, no. 7, pp. 86–94, 2014.
[45] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.
[46] M. Liu, "Ai-challenger-weather-forecast," 2019. [Online]. Available: https://github.com/Ericonaldo/AI-Challenger-Weather-Forecast
[47] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen, and F. Petitjean, "Characterizing concept drift," *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–994, 2016.



Ming Zhou received a M.S. degree in computer application technology from the School of Computer Science and Information Engineering, Hefei University of Technology, Anhui, China, in 2020. He is a PhD candidate in the Faculty of Engineering and Information Technology, University of Technology Sydney, Australia. He is a Member of the Decision Systems and e-Service Intelligence Lab, Australia Artificial Intelligence Institute, University of Technology Sydney. His research interests include concept drift and non-stationary stream learning.



Jie Lu (F'18) is an Australian Laureate Fellow, IFSA Fellow, Distinguished Professor and the Director of Australian Artificial Intelligence Institute (AAII) at the University of Technology Sydney, Australia. She received a PhD degree from Curtin University in 2000. Her main research expertise is in transfer learning, concept drift, fuzzy systems, decision support systems and recommender systems. She has been awarded 12 Australian Research Council (ARC) discovery and linkage grants and led 20 industry projects. She has published over 500 papers in IEEE Transactions and other leading journals and conferences. Many of her research results have been successfully transferred to industry. She has delivered 40 keynote speeches at international conferences, supervised 50 PhD students to completion, serves as Editor-In-Chief for Knowledge-Based Systems and received national and international awards including the IEEE Transactions on Fuzzy Systems Outstanding Paper Awards (2018 and 2022).



Yiliao Song received a M.S. in probability and statistics in mathematics from the School of Mathematics and Statistics, Lanzhou University, China, in 2015. She received a Ph.D. degree in computer science from the University of Technology Sydney, Australia. Her research interests include concept drift, data stream mining and real-time prediction. She is a research fellow at RMIT Enterprise AI and Data Analytics Hub, RMIT University, Australia.



Guangquan Zhang is an Australian Research Council (ARC) QEII Fellow, Associate Professor and the Director of the Decision Systems and e-Service Intelligent (DeSI) Research Laboratory at the Australian Artificial Intelligence Institute, University of Technology Sydney, Australia. He received his PhD in applied mathematics from Curtin University, Australia, in 2001. From 1993 to 1997, he was a full Professor in the Department of Mathematics, Hebei University, China. His main research interests lie in fuzzy multi-objective, bilevel and group decision making, fuzzy measures, transfer learning and concept drift adaptation. He has published six authored monographs and over 500 papers including some 300 articles in leading international journals. He has supervised 40 PhD students to completion and mentored 15 Postdoc fellows. Prof. Zhang has won ten very competitive ARC Discovery grants and many other research projects. His research has been widely applied in industries.